

10/10 Answer to “Set Goals for Claude Code”

Applying the AI Analytical Framework to Anthropic’s Fastest-Growing Product



RIDHIMA KHURANA

FEB 27, 2026 • PAID

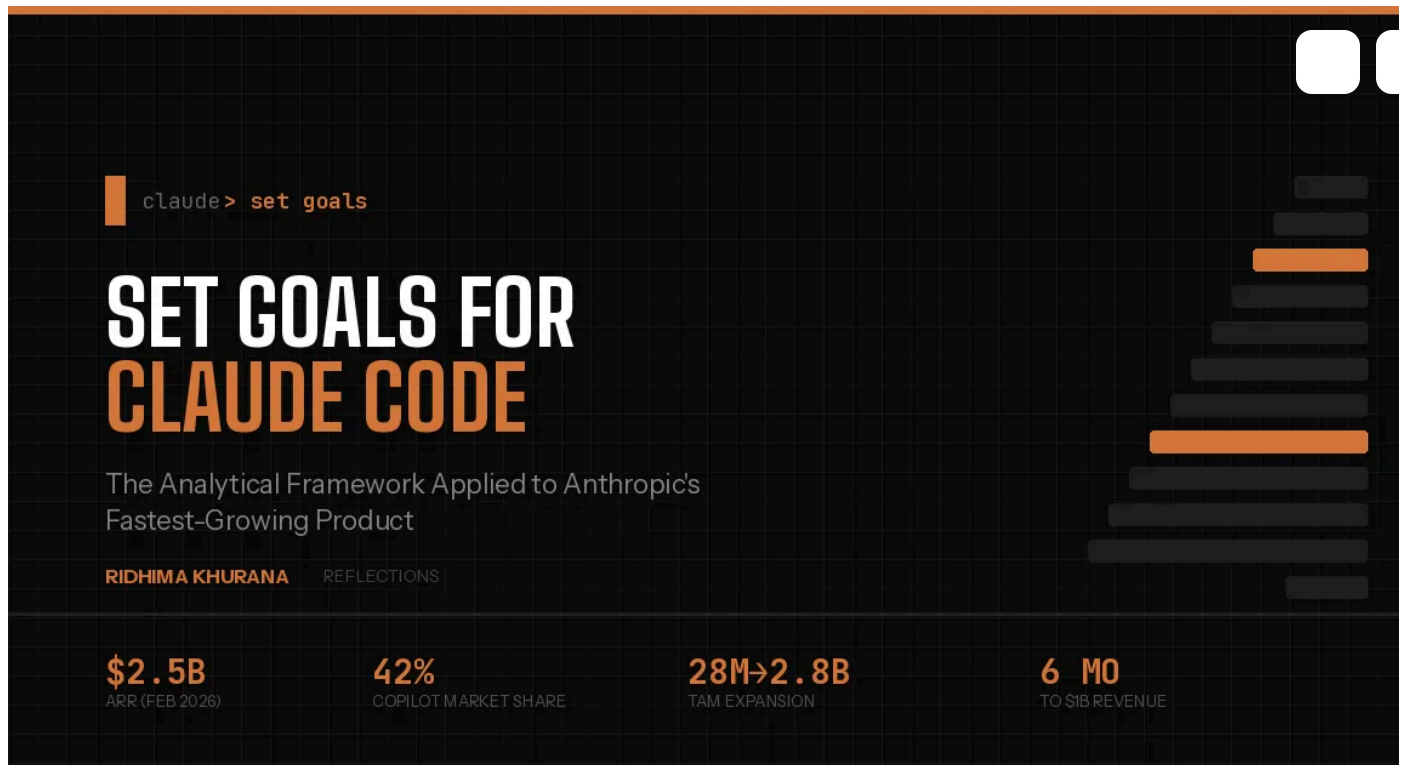


9



2

Share



Last time, I walked through my complete analytical framework using an OpenAI HomePod question. (If you missed it, go read that first. Everything I’m about to do builds on that structure.)

Today: same framework, completely different scenario.

“Set goals for Claude Code.”

This question could show up in an Anthropic loop, but honestly, it could show up anywhere. Claude Code is the product every AI PM should be able to reason about

right now. It hit \$1 billion in annualized revenue six months after launch. Faster than ChatGPT's revenue ramp. By February 2026, estimates place it near \$2.5 billion ARR. The creator of the product is getting recognized at airports like he's a minor celebrity. For a *command line tool*.

And yet, most candidates would still walk into this interview and say something like "I'd track DAU and weekly retention." Which is like saying you'd measure the success of the iPhone by counting how many people turned it on.

Let's do better.

Step 1: Break the Prompt

Before you say a single word about metrics, slow down and interrogate the question itself. This is where interviewers separate the reactive candidates from the thoughtful ones.

Three things to notice immediately:

Claude Code is a terminal-native agent, not a chatbot with a code mode. This distinction matters enormously. GitHub Copilot autocompletes your lines. Cursor gives you a smart IDE. Claude Code lives in the terminal, has full file system access and operates autonomously. You tell it "refactor the authentication system across all services," walk away, come back, and the work is done. That's not code completion. That's delegation. The UX expectations, trust dynamics, and success criteria are fundamentally different when your user is handing over the steering wheel.

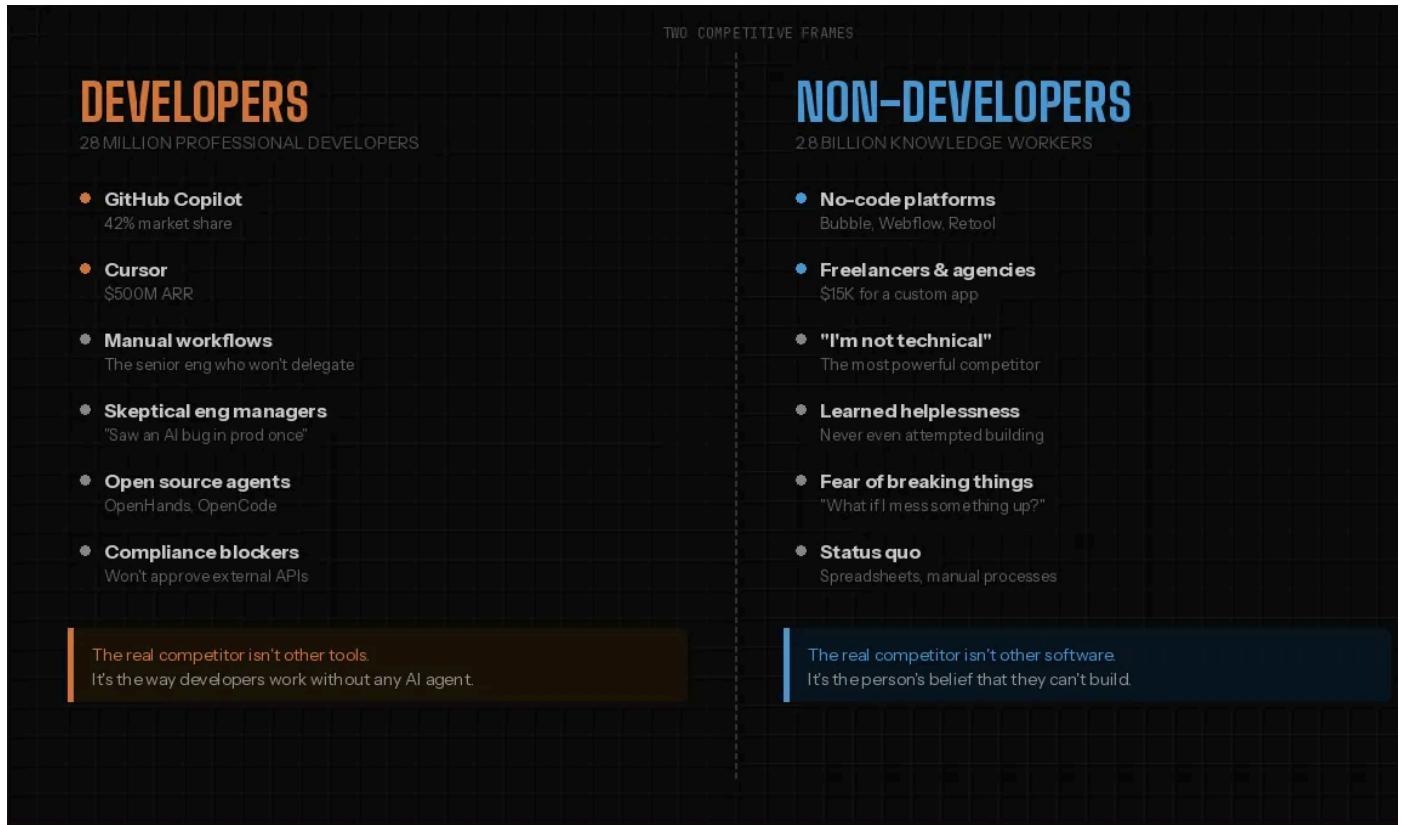
"Code" is becoming a misnomer, and that's the real strategic insight here. During 2025 holiday break, Claude Code went viral because people with *zero* programming experience used it to build apps, personal websites, and tools from scratch. The creator of Claude Code, Boris Cherny, said he hasn't manually edited a single line of

code since November 2025. He predicts that by end of 2026, the title “software engineer” starts getting replaced by “builder.” Then Anthropic launched Cowork for non-technical users in January 2026. So when someone says “set goals for Claude Code,” you need to immediately ask: are we setting goals for a developer tool, or for universal builder that also serves non-developers?

The answer, and this is where most candidates miss the real opportunity, should be both. The 10/10 answer recognizes that Claude Code’s near-term dominance comes from developers, but its long-term TAM explosion comes from a specific audience, call them the “Builder Audience” that includes freelancers, solopreneurs, digital marketers, etc who are not afraid to learn something new. Your goals need to serve customer segments simultaneously without diluting either.

This product is Anthropic’s revenue engine. In a company generating roughly \$9 billion in total revenue, Claude Code accounts for a massive and rapidly growing share. It not just a feature but the growth story investors are underwriting at a \$380 billion valuation. Your metrics need to reflect that weight.

Step 2: Who Are You Really Competing With?



Here's where most candidates go wrong. They say "GitHub Copilot and Cursor." That's half the game. Because if we're serious about the non-developer segment, the competitive landscape splits wide open.

For **developers**, the named competitors are real. Copilot still holds roughly 42% market share among paid AI coding tools and benefits from Microsoft's distribution machine. Cursor crossed \$500 million ARR and has a devoted following. In a UC San Diego/Cornell survey of 99 professional developers in January 2026, Claude Code, Copilot, and Cursor were essentially neck and neck in adoption.

But the deeper competitive frame in the developer segment isn't other tools.

It's the way developers currently work without any AI agent at all.

The biggest competitor is the senior engineer who still does everything manually because they don't trust any tool enough to delegate. It's the engineering manager who saw a colleague's AI-generated code introduce a subtle bug in production and decided

the whole category wasn't ready. It's the compliance team that won't approve any tool that sends code to an external API. And below all of that, the open-source movement (OpenHands, OpenCode) is growing specifically because developers want the agent workflow but bristle at sending proprietary code through a third-party API.

For non-developers, the competitive frame is entirely different. Here, Claude Code isn't competing with Copilot or Cursor. It's competing with:

1. *No-code platforms* like Bubble, Webflow, and Retool, which let non-technical people build apps through visual interfaces. These tools are popular precisely because they removed the need to write code. Claude Code's pitch to this segment is even bolder: you don't need a visual drag-and-drop interface either. Just describe what you want in plain English.
2. *Freelancers and agencies.* A small business owner who needs a custom inventory system currently hires a developer on Upwork or pays an agency \$15,000. Claude Code's value proposition is: skip the middleman entirely. Describe what you need and the tool builds it. That's a fundamentally different competitive frame than "which IDE extension is best."
3. *The person's own belief that they can't build software.* This is the most powerful competitor of all, and it's invisible in any market share report. Millions of people with genuine software ideas simply never attempt them because "I'm not technical." Claude Code's holiday viral moment proved that barrier can be broken. But breaking it at scale requires intentional product and go-to-market work, not just happy accidents during winter break.

Say all of that in an interview, and you'll see the interviewer sit up. You've just demonstrated that you understand two completely different value propositions living inside one product.

Step 3: Why Is Anthropic Investing in This?

Three layers. A strong candidate articulates all three.

Distribution for the model layer. Models are commoditizing fast. GPT-5.2 is strong Gemini is strong. The moat isn't the model. It's the surface area through which people interact with the model every day, building workflow dependency that makes switching painful. Claude Code locks in developers. Cowork locks in knowledge workers. Two flywheels, both spinning in Anthropic's direction.

The data flywheel, now with two fuel sources. Developer sessions generate messy, real-world training signal that competitors can't replicate from behind a benchmark. But here's what most candidates miss: the non-developer signal is arguably *more* valuable. When a restaurant owner says "build me a reservation system that texts customers 2 hours before their booking and lets them cancel by replying STOP," the prompt encodes business logic, UX intuition, and domain expertise that no developer-only dataset captures. Anthropic is building a corpus of how non-technical humans think about software. That data moat is unique and compounding.

TAM expansion from 28 million to 2.8 billion. There are roughly 28 million professional developers worldwide. There are 2.8 billion knowledge workers. If even 5% become "builders" using agentic tools, that's 140 million users, 5x the entire developer population. Without the non-developer segment, Anthropic is a very successful developer tools company. With it, they're building the next platform. That's the difference between a \$50 billion outcome and a \$380 billion one.

Step 4: When Do Users Derive Value?

Value looks different for each segment, and you need to articulate both.

For developers, value arrives at the moment a delegated task is completed correctly and the developer trusts the result enough to ship it without a full manual review. Not when they type a prompt. Not when Claude Code starts generating files. When the

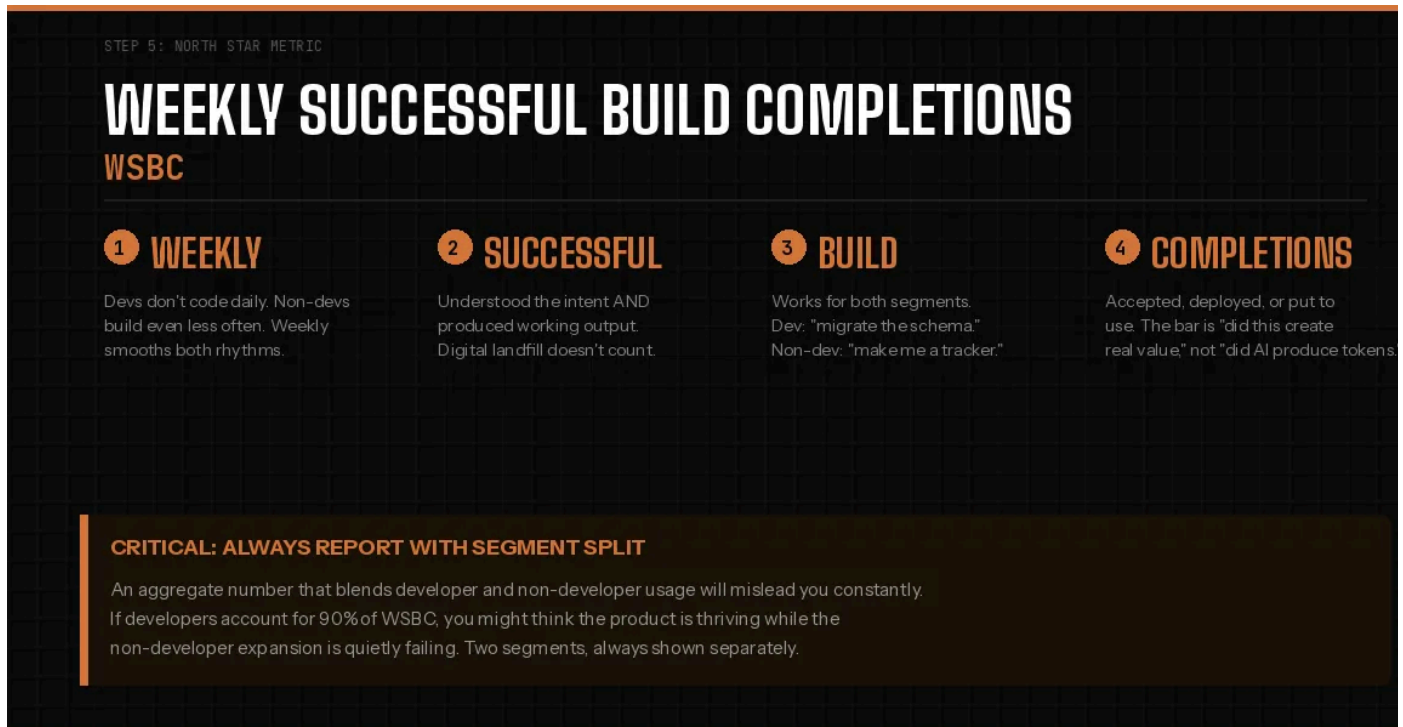
loop is closed: the refactor is done, the tests pass, the code is clean enough that the developer merges it. The key insight is the word *trust*. If developers routinely review every single line with the same scrutiny they'd apply to their own code, the product hasn't actually saved time. It's moved the work from "writing" to "reviewing." The real value inflection happens when the developer spot-checks rather than line-audits.

For non-developers, value arrives at a completely different moment: when they have a working thing they can use or show someone, and they didn't need to understand how it was built. A restaurant owner who describes a reservation system and gets a working prototype they can click through, share with their staff, and iterate on with follow-up prompts? That's magic. They don't care if the code is clean. They don't care if it's React or vanilla JavaScript. They care that it *works* and that they built it themselves (with an AI, sure, but still). The emotional component here matters. The feeling of "I made this" is powerful and sticky in a way that developer productivity gains are not. Non-developers who successfully build something become evangelists. Developers who save 30 minutes on a refactor just quietly keep using the tool.

The implication for goal-setting: your North Star needs to capture *both* of these valuable moments, or you need two complementary North Stars with a clear hierarchy.

Step 5: The North Star Metric

Weekly Successful Build Completions (WSBC): The count of tasks per week where Claude Code autonomously completed the user's intent and the output was accepted, deployed, or put to use, without the user needing to start over or abandon the result.



I’m deliberately using “Build Completions” rather than “Task Completions” because captures both segments:

Weekly — Developers don’t use Claude Code every day. Some days are meetings, some are planning, some are deep-focus manual work. For non-developers, usage is even more sporadic. They build when they have a need, not on a daily schedule. Weekly smooths out both rhythms without lagging so far behind that you can’t course-correct.

Successful — Claude Code understood the intent AND produced output that actually works. For developers, this means the code passes tests and meets requirements. For non-developers, this means the thing does what they asked it to do. A tool that generates 50 files nobody can use isn’t succeeding. It’s generating digital landfill.

Build Completions — This is the key framing choice. A “build” is a meaningful unit of creation: for developers, that’s “add authentication to this endpoint” or “migrate the database schema.” For non-developers, it’s “make me a budget tracker” or “build a landing page for my bakery.” The word “build” works across both segments in a way that “task” or “code commit” doesn’t.

Accepted or put to use — The competitive bar. For developers, this means merged, committed, or deployed without a full rewrite. For non-developers, this means they actually used the output, shared it, or iterated on it rather than closing the tab in frustration. The bar is “did this create real value” not “did the AI produce tokens.”

Critical: WSBC must *always* be reported with a segment split. An aggregate number that blends developer and non-developer usage will mislead you constantly. If developers account for 90% of WSBC, you might think the product is thriving while the non-developer expansion is quietly failing. Two segments, always shown separately, with the aggregate as a headline.

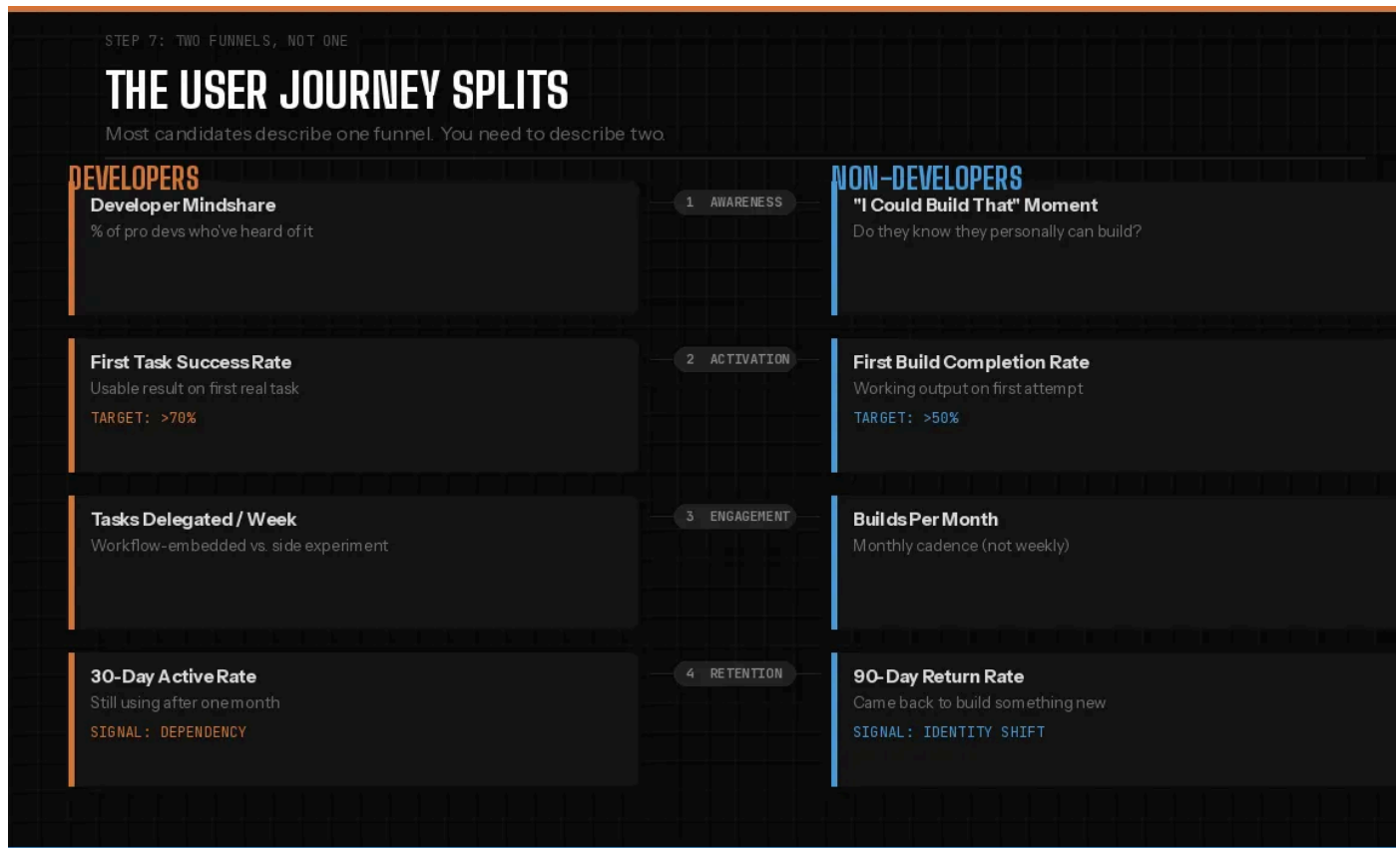
Step 6: Ecosystem Impact Metrics

Developer → Platform Deepening Rate: The percentage of Claude Code developer users who expand into Anthropic’s API (building products powered by Claude) within 60 days. A developer who uses Claude Code to build their own product, then integrates Claude’s API into that product for their end users, has gone from *customer* to *distribution partner*. That’s the most valuable conversion in Anthropic’s entire funnel.

Non-Developer → Repeat Builder Rate: The percentage of non-developer users who complete a second build within 30 days of their first. The first build might be curiosity or a viral moment. The second build means they’ve internalized the idea that they can create software. That mental shift is the real product-market fit signal for this segment.

Cross-Product Expansion Rate: The percentage of Claude Code users (both segments) who increase their usage of other Anthropic products (Claude.ai, Cowork, API) within 30 days. If Claude Code is growing WSBC but users never touch another Anthropic product, it’s a successful standalone tool but a strategic underperformer. The whole point is that Claude Code is the gateway.

Step 7: Secondary Metrics — The User Journey



The user journey splits here, and the split is where most candidates lose points. They describe one funnel. You need to describe two.

Developer Funnel

Awareness: Developer Mindshare. What percentage of professional developers have heard of Claude Code? Awareness is high among AI-forward developers but likely thinner in enterprise IT shops, non-US markets, and non-English-speaking developer communities. The question is how fast awareness converts outside the early adopter bubble.

Activation: First Task Success Rate. Of developers who install Claude Code and run their first substantive task (not “hello world”), what percentage get a usable result?

Developer tools get one shot. A developer who tries Claude Code on a real task and gets garbage output will uninstall it and tell five friends. Target: above 70%.

Engagement: Tasks Delegated Per Week. Once activated, how many meaningful tasks does a developer delegate per week? This measures whether Claude Code is becoming part of the daily workflow or is a "I'll try it when I have time" side experiment. Workflow-embedded tools retain. Side experiments churn.

Retention: Monthly Active Developer Rate. Among developers who completed at least one successful task, what percentage are still using Claude Code 30 days later? Developer retention is about *dependency*. A developer who has restructured their workflow around delegation needs the tool. That's the signal you want.

Non-Developer Funnel

Awareness: "I Could Build That" Moment. This isn't traditional awareness (have you heard of Claude Code?). For non-developers, the relevant awareness metric is: do they know that *they personally* can build software using AI? The holiday viral moment created millions of these moments organically. The question is whether Anthropic can manufacture them at scale through marketing, partnerships, and word of mouth.

Activation: First Build Completion Rate. Of non-developers who try Claude Code (or Cowork) for the first time, what percentage end up with something that works? This is the single most important metric for the non-developer segment. The failure modes are different from developers: non-developers don't get frustrated by bad code quality; they get frustrated by *not knowing what to ask for next* when something doesn't work. The activation challenge is conversational, not technical. Target: above 50% (lower than developers because the learning curve is steeper in a different way).

Engagement: Builds Per Month. Non-developers build less frequently than developers. A restaurant owner doesn't need a new tool every week. But they might need one every month (a reservation system in January, a staff scheduling tool in February, a

customer feedback form in March). Monthly build frequency is the right cadence to track.

Retention: 90-Day Return Rate. Did the non-developer come back to build something new within 90 days? If yes, they've crossed from "tried an AI thing once" to "this is how I solve problems now." That identity shift is the holy grail.

Step 8: Guardrail Metrics

Guardrails prevent you from celebrating a North Star that's lying to you.

Quality (Developer Segment)

Bug Introduction Rate. If Claude Code completes a task but introduces a subtle bug that isn't caught until production, that's worse than not using the tool at all. Must be segmented by code complexity: a bug in a CRUD endpoint is embarrassing; a bug in a payment processing flow is a lawsuit waiting to happen. Target: at or below the baseline rate for experienced human developers on equivalent tasks.

Code Review Override Rate. What percentage of Claude Code's output gets substantially changed during code review? A high override rate means the tool generates plausible-looking code that experienced developers don't trust. Dangerous territory, because junior developers might not catch the same issues.

Vulnerability Introduction Rate. In February 2026, Anthropic launched Claude Code Security after finding over 500 high-severity vulnerabilities in open-source codebases. Impressive capability. But the inverse question: how often does Claude Code generate code with vulnerabilities? If your AI agent introduces SQL injection at the same speed it introduces features, you've created a problem that scales with adoption.

Quality (Non-Developer Segment)

Build Abandonment Rate. What percentage of non-developer sessions end with the user giving up before they have a working output? High abandonment is the loudest signal that the non-developer experience isn't working. Unlike developers who can debug error messages, non-developers hit a wall and leave. They don't file bug reports. They just stop.

"Works But Shouldn't Ship" Rate. Non-developers can't evaluate code quality. They see a working prototype and assume it's production-ready. If Claude Code builds a customer-facing app for a small business that has no authentication, no input validation, and stores passwords in plain text, the *user* thinks it's great. Until it isn't. This is a unique guardrail for the non-developer segment: you need to measure how often the tool produces output that looks functional but has hidden risks the user can't evaluate.

Support Escalation Rate. Non-developers will generate a disproportionate share of support tickets relative to developers. If this rate spikes, it signals that the product is attracting non-developers but not adequately serving them, which is worse than not attracting them at all. A frustrated non-developer who had a bad first experience will tell everyone that AI coding tools "don't really work."

Cannibalization

Net Claude Ecosystem Usage Lift. Does Claude Code usage increase overall time spent with Claude products, or does it just shift usage from Claude.ai to Claude Code? You want to expand the pie, not rearrange the slices.

Partner Ecosystem Health. Claude's models power other tools (Cursor uses Claude Sonnet, many IDE extensions use Claude's API). If Claude Code's growth comes at the expense of API revenue from these partners, that's a complex strategic trade-off we're tracking.

Cowork vs. Claude Code Cannibalization. This is the new one that's specific to the dual-segment strategy. Anthropic launched Cowork in January 2026 explicitly for non-technical users. If non-developers are being directed to Claude Code when they should be in Cowork, or vice versa, both products suffer. You need to measure whether the two products are expanding the total user base or splitting the same users into two buckets with worse experiences in each.

Step 9: Where Does Bias Hide?

STEP 9: WHERE DOES BIAS HIDE?

YOUR METRICS ARE LYING TO YOU

Two segments means bias hides in twice as many places.

Language & Framework Bias DEVELOPER Python/JS users thrive. Go/COBOL users churn. Aggregate WSBC hides the gap. Your data says 'product-market fit' Reality says 'training data fit'	Complexity Bias NON-DEVELOPER Simple builds succeed. Complex ones fail. WSBC looks great on easy wins. You'll expand marketing too early and watch abandonment spike.
Survivorship Bias BOTH You only see people who stuck around. The dev who tried once and left is invisible. Data shaped by success makes the product look better than it is.	Demographic Bias NON-DEVELOPER Holiday viral users: tech-curious, English-speaking, free time to experiment. Like testing with Tesla buyers and concluding it's ready for mass market.

This is where you separate yourself from every other candidate. Most people stop at guardrails. Go deeper.

Does bias exist in our North Star? Yes. And with two segments, it hides in twice as many places.

Language and framework bias (developer segment). If Claude Code performs significantly better on Python and JavaScript (the languages with the most training

data) than on Rust, Go, or COBOL, your WSBC is systematically overstating product market fit. A developer working on a Go microservice who gets mediocre output with churn, but your aggregate metrics look great because the Python users are thriving.

Complexity bias (non-developer segment). Non-developers who succeed will skew toward simple builds: landing pages, basic CRUD apps, simple automations. Your WSBC will look fantastic because these are easy wins. But if you're measuring "build completed" without segmenting by complexity, you might conclude the product is ready for more ambitious non-developer use cases (building an actual SaaS product creating a complex workflow automation) when it isn't. You'll expand marketing, draw more ambitious non-developers to the product, and watch abandonment rates spike.

Survivorship bias in both segments. You only measure people who stuck around long enough to complete a build. The developers who tried Claude Code, got a bad result and never came back don't show up in your WSBC. The non-developers who installed it, stared at a terminal prompt for 30 seconds, and thought "this isn't for me" are invisible in your funnel metrics. Your data is shaped by the people who succeeded, which makes the product look better than it is for the broader population you're trying to serve.

Demographic bias in the non-developer segment. The holiday viral moment that brought non-developers to Claude Code likely attracted a specific demographic: tech-curious, English-speaking, with enough disposable time during winter break to experiment with AI tools. Generalizing from this cohort to "all non-developers" would be like testing a product with early Tesla buyers and concluding it's ready for the mass market.

When does it emerge?

At every expansion decision, but especially at the developer-to-non-developer crossover. "Is the product ready for a non-developer marketing push?" "Should we invest in non-English language support?" "Do we need a fundamentally different U

for non-developers, or can we smooth the edges of the existing terminal?" Each of these decisions will be informed by metrics trained on a biased sample.

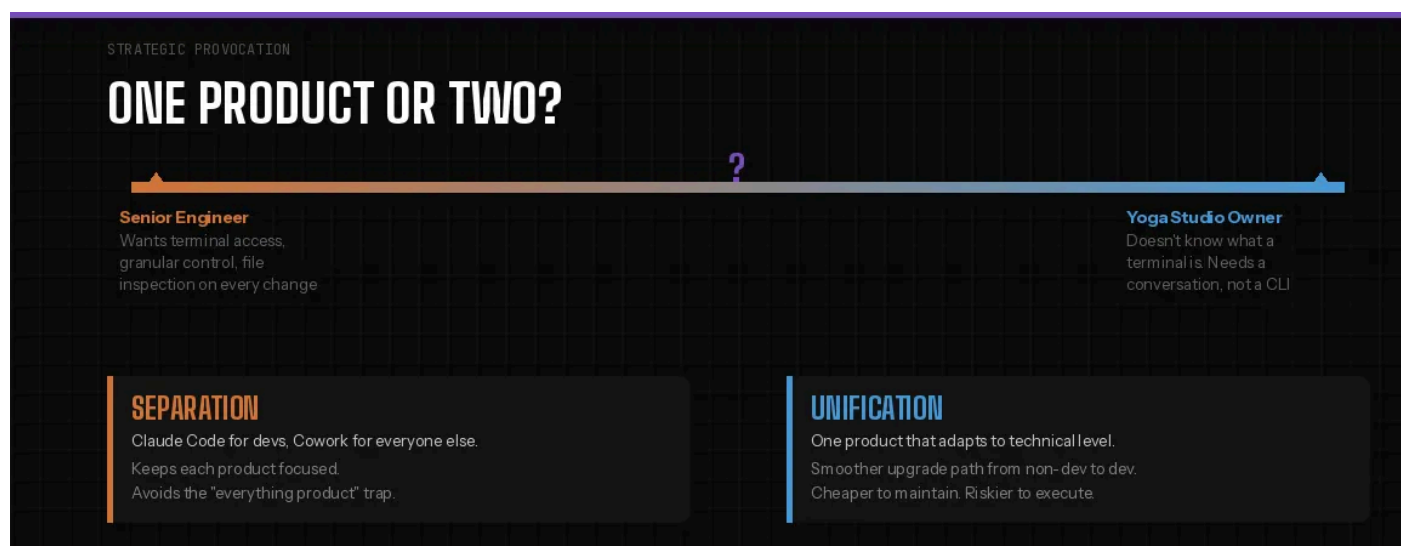
What shape does it take?

In the developer segment: false negatives in underserved languages and frameworks; false positives in the Python/JS core.

In the non-developer segment: false positives on simple builds that mask inability to handle complex ones. False negatives in demographics and geographies that weren't part of the viral moment.

The fix: Segment every metric by programming language, user seniority level, and codebase complexity for developers. Segment by build complexity, technical background, and acquisition channel for non-developers. Report developer and non-developer metrics separately, always. Don't wait until you have a scaling problem. Build the segmentation infrastructure before you launch the non-developer growth push.

A Strategic Provocation: One Product or Two?



One more thing worth raising in the interview, because it shows you think beyond metrics question.

There's a fundamental tension in the dual-segment strategy. The developer experience and the non-developer experience may need to diverge more than they currently do. Anthropic has started to address this by launching Cowork as a separate product for non-technical users. But the line between Claude Code and Cowork is still blurry.

Consider the spectrum:

At one end, you have a senior engineer who wants Claude Code to refactor a distributed system while preserving backward compatibility. They want granular control, terminal access, and the ability to inspect every file change.

At the other end, you have a yoga studio owner who wants to build an online booking system. They don't know what a terminal is. They need a conversational interface that asks them questions ("Should clients be able to cancel within 24 hours? Do you want automatic reminders?") and produces something they can use without ever seeing a line of code.

These are fundamentally different products wearing similar branding. The question is should Anthropic keep them as separate products (Claude Code for developers, Cowork for everyone else), or should Claude Code evolve into a single product that adapts its interface based on the user's technical sophistication?

There are good arguments for both. Separation keeps each product focused and avoids the "everything product" trap that dilutes both experiences. Unification creates a smoother upgrade path (non-developer starts with simple builds, gradually takes on more control, eventually becomes a developer-level user) and is cheaper to maintain.

Boris Cherny's prediction that the title "software engineer" gets replaced by "builder" suggests the ultimate destination is one unified tool. If the distinction between "developer" and "non-developer" dissolves, having two separate products is a liability.

But if the transition takes a decade instead of two years, running a single product tries to serve both segments risks serving neither well.

Raising this tension in an interview shows you understand resource allocation, product-market timing, and platform strategy simultaneously. That's the difference between a candidate who can define metrics and a candidate who can drive growth.

If you're preparing for AI product interviews at OpenAI, Google, Anthropic, Meta, or other tech companies, I coach a small number of candidates each quarter. Reach out to me to craft your dream role.



9 Likes • 2 Restacks

Discussion about this post

Comments Restacks



Write a comment...

